

一、 操作使用说明

详见《基于 DMD 的共聚焦显微成像系统软件操作说明.docx》2.5

二、 配置文件说明

详见《基于 DMD 的共聚焦显微成像系统软件操作说明.docx》2.5

三、 外引用库说明

只有 ConfocalUILib 需要引用外部库	
openGL	glaux.lib, GLU32.Lib, OpenGL32.Lib glu32.dll, opengl32.dll 通过以下位置拷贝 VS 的 SDK 库: D:\Windows Kits\10\Include\10.0.17763.0\um\gl
特别库	legacy_stdio_definitions.lib 因为 vs2017 默认编译时将许多标准库采用内联方式处理, 因而没有可以链接的标准库文件, 所以要专门添加标准库文件来链接标准库中的函数 参考 https://blog.csdn.net/gongxun1994/article/details/81813874
Vtk 库	122 个库

只有 ConfocalUILib 需要引用外部源文件	
ChartClass	绘制曲线
Global	HGloableFunction.cpp HImgProcess.cpp HTimer.cpp

引用的 vtk 库	
Win32-Debug Win32-Release x64-Release	
X64-Debug	

默认需要加的引用

vtkGUISupportMFC-8.2d.lib

vtkInteractionStyle-8.2d.lib

vtkRenderingOpenGL2-8.2d.lib
vtkRenderingCore-8.2d.lib
vtkCommonDataModel-8.2d.lib
vtkCommonExecutionModel-8.2d.lib
vtkCommonCore-8.2d.lib
vtkFiltersGeneral-8.2d.lib
vtkFiltersGeometry-8.2d.lib

默认需要加的库：

vtkCommonColor-8.2d.lib
vtkCommonComputationalGeometry-8.2d.lib
vtkCommonCore-8.2d.lib
vtkCommonDataModel-8.2d.lib
vtkCommonExecutionModel-8.2d.lib
vtkCommonMath-8.2d.lib
vtkCommonMisc-8.2d.lib
vtkCommonSystem-8.2d.lib
vtkCommonTransforms-8.2d.lib
vtkFiltersCore-8.2d.lib
vtkFiltersExtraction-8.2d.lib
vtkFiltersGeneral-8.2d.lib
vtkFiltersGeometry-8.2d.lib
vtkFiltersSources-8.2d.lib
vtkFiltersStatistics-8.2d.lib
vtkglew-8.2d.lib
vtkGUISupportMFC-8.2d.lib
vtkImagingCore-8.2d.lib
vtkImagingFourier-8.2d.lib
vtkInteractionStyle-8.2d.lib
vtkRenderingCore-8.2d.lib
vtkRenderingOpenGL2-8.2d.lib
vtksys-8.2d.lib

x64-Debug 版的引用，多 2 个

vtkRenderingFreeType-8.2d.lib
vtkRenderingGL2PSOpenGL2-8.2d.lib

x64-Debug 版，多 6 个 dll

vtkGUISupportMFC-8.2d.lib
vtkInteractionStyle-8.2d.lib
vtkRenderingOpenGL2-8.2d.lib
vtkRenderingCore-8.2d.lib
vtkCommonDataModel-8.2d.lib

vtkCommonExecutionModel-8.2d.lib
vtkCommonCore-8.2d.lib
vtkFiltersGeneral-8.2d.lib
vtkFiltersGeometry-8.2d.lib
vtkfreetype-8.2d.lib
vtkgl2ps-8.2d.lib
vtkpng-8.2d.lib
vtkRenderingFreeType-8.2d.lib
vtkRenderingGL2PSOpenGL2-8.2d.lib
vtkzlib-8.2d.lib

四、 IDE 属性配置

ConfocalCore 的以下属性，在不同编译配置都一样：

- ➔ “VC++目录” —> “包含目录”；
- ➔ “VC++目录” —> “库目录”；
- ➔ “C/C++” —> “常规” —> “附加包含目录”
- ➔ “链接器” —> “附加库目录”；
- ➔ “链接器” —> “输入” —> “附加依赖项”；
- ➔ “生成事件” —> “生成后事件” —> “命令行”；

ConfocalScannerLib 的以下属性，在不同编译配置都一样：

- ➔ “VC++目录” —> “包含目录”；
- ➔ “VC++目录” —> “库目录”；
- ➔ “C/C++” —> “常规” —> “附加包含目录”
- ➔ “链接器” —> “附加库目录”；
- ➔ “链接器” —> “输入” —> “附加依赖项”；
- ➔ “生成事件” —> “生成后事件” —> “命令行”；

ConfocalUILib 的以下属性，在不同编译配置都一样：

- ➔ “VC++目录” —> “包含目录”；
- ➔ “VC++目录” —> “库目录”；
- ➔ “C/C++” —> “常规” —> “附加包含目录”
 - 头文件不一样，vtk 的 win32 需要的头文件比 x64 多很多！
- ➔ “链接器” —> “附加库目录”；
- ➔ “链接器” —> “输入” —> “附加依赖项”；
- ➔ “生成事件” —> “生成后事件” —> “命令行”；

DMDCConfocal 的以下属性，在不同编译配置都一样：

- ➔ “VC++目录” —> “包含目录”；
- ➔ “VC++目录” —> “库目录”；
- ➔ “C/C++” —> “常规” —> “附加包含目录”
- ➔ “链接器” —> “附加库目录”；
 - 路径不一样
- ➔ “链接器” —> “输入” —> “附加依赖项”；
 - Win32 需要的是：\$(SolutionDir)\$(Configuration)\；
 - x64 需要的是：\$(SolutionDir)\$(PlatformName)\\$(Configuration)\；
- ➔ “生成事件” —> “生成后事件” —> “命令行”；

PS：灰色底的部分代表使用程序默认值，没有修改过。

五、 接口结构说明

头文件汇总	
CarlVideo.h	VideoTester 相机相关接口 ImgProcessTaster Notify 的消息从 3000 开始
HConfFile.h	StageTester
HConfocalCore.h	
HConfocalPlug.h	
HConfocalUI.h	
HCoreProcess.h	ImgProcessTaster Notify 的消息从 5000 开始
HGearBox.h	StageTester Notify 的消息从 4000 开始
HGloableFunction.h	公共函数
HImgProcess.h	公共函数
HTimer.h	StageTester, 公共函数
IDMDManager.h	WLPDmdTest Notify 的消息从 6000 开始
IHsmObserver.h	VideoTester 相机相关接口 ImgProcessTaster StageTester WLPDmdTest
IPatternGenerate.h	WLPDmdTest
ptnSingleton.h	
RenderChain.h	ImgProcessTaster

RenderChain.h , 渲染链列表	
class HVideoRenderChain : public HVideoRender,public list<HVideoRender*>	
本身继承了 list 集合列表, 本身会去维护加载进来的所有渲染对象	
Render(HVideoHeader* pHeader,LPBYTE pBuffer)	轮询集合内的渲染对象, 每个都 (*pos)->Render(pHeader,pBuffer);
Find(HVideoRender* key)	返回查找到的渲染对象, 返回索引 list<HVideoRender*>::iterator pos
1、 主动渲染对象: 获取渲染链, 直接调用渲染链的 Render (...) 函数; 2、 被渲染对象: 通过获取这个渲染链, 然后.add, 增加进去;	

HConfocalCore.h, 核心库接口		
	各类 Notify 消息	
	class HCorePanel: IHsmObserver, IHsmSubject	面板基类【对应 ConfocalUILib】 用于 View、停靠面板、对话框
	Init(LPVOID p_Param)	
	UnInit()	
	Wnd* GetCWnd()	得到窗体
	int OnSubjectNotified	
	HConfocalPlug: IHsmObserver, IHsmSubject	插件基类【ConfocalScannerLib】
	Set(void* WParas)	设置参数
	Start(void* vParas=0)	启动线程
	Stop()	管边线程
	InitPlugin(LPVOID p_Param=0)	初始化
	UnInitPlugin()	卸载
	GetPluginName()	获取名称
	HVideoRender* GetVideoRender()	获取渲染链
	int OnSubjectNotified(...)	通知回调
	HConfocalCore: IHsmObserver, IHsmSubject	核心基类
	RenderChainType	不同的渲染类型（共 3 种） Normal、Confocal、SL
	Init(LPVOID p_param=0)	初始化
	Uninit(LPVOID p_param=0)	卸载
	GetDockablePanel(DOCKPANEL_TYPE)	获取停靠面板
	GetViewPanel(VIEWPANEL_TYPE)	获取 View 面板
	HCorePanel* GetMsgPanel(MESSAGE_TYPE)	传入 bModal , 获取对话框
	HConfocalPlug* GetPlugin(PLUGIN_TYPE)	获取插件（即线程）
	HCoreProcess* GetCoreProcess	获取算法
	HEasyFunction* GetFunction()	获取一些公用函数
	InitLog()	初始化日志
	LogString(CString strMsg)	打印日志
	CloseLog()	关闭日志
	IDMDManager* GetDmdManager()	获取 DMD
	HVideoDevice* GetCurVideo()	获取当前的 Video
	HVideoManager* GetVideoManager()	获取 Video 的 Manager
	HGearBox* GetGearBox()	获取载物台
	HConfigure* GetConfigure()	获取配置接口
	HVideoRenderChain* GetRenderChain(RenderChainType)	获取指定的渲染链列表

HConfocalPlug.h, 插件基类，即线程	
	PLUGIN_TYPE 的枚举，含 MAP、3DScanner……、PLUGIN_DIFFMEASURE

HConfocalUI.h, 面板基类，即对话框

	DOCKPANEL_TYPE	停靠类枚举
	VIEWPANEL_TYPE	视图类枚举
	MESSAGE_TYPE	对话框类枚举

六、 独立工程/库说明

6.1、 核心库 ConfocalCore

Config	
Confocal_Config.h	弃用的配置文件读取?
LogLib	
LogLib.h	读取时间, 打开 txt 进行日志写入
Splash	
SplashWnd.h	CSplashWnd 类, 没有指向类
SplashPanel.h	SplashPanel 类, 指向 IDD_DLGLFLASH
头文件源文件	
ConfocalCore.h	程序接口
ConfocalCoreModule.h	核心基类的实例化
	该程序里面的两个库 CConfocalUILib m_ConfocalUILib; CConfocalScannerLib m_ScannerLib;
	动态图像处理库 HCoreProcessMg* mProcessMg;//图像处理动态库
	动态相机库: 通过 mVideoType 去获取 Video 的 Manager HVideoType* mVideoType;//所有相机类 HVideoManager* m_ConfocalVideoManage; HVideoDevice* m_CurVideoDevice;//当前相机设备
	动态 DMD 库 IDMDManager* m_DmdManager; WlpDMDParas mWlpDMDParas;
	动态的载物台库 HGearBox* m_GearBox;
	三个渲染链 HVideoRenderChain m_RenderChain; HVideoRenderChain m_ConfocalChain; HVideoRenderChain m_SLChain; ConfocalChain、SLChain 应该是被其他模块获取, 再去绑定! RenderChain 直接在 Init()函数中直接被相机绑定 “m_CurVideoDevice->SetRender(&m_RenderChain);”

初始化 1

创建初始化界面

根据 INI 配置文件，去初始化相机实例、载物台实例、DMD 实例、算法实例；

查找到了相机，就去打开相机，OpenCamera()存在问题？

初始化任务实例

初始化 2

创建 UI 库所有实例；

如果有相机，绑定各自的渲染链；

6.2、 界面库 ConfocalUILib

- ➔ 停靠的面板需要资源界面；
- ➔ 视图都是直接 Create 创建的，没有资源界面；
- ➔ 差动的视图是一个 tab 界面，创建完之后，里面有一个 tab 界面

6.2.1、 资源解析

对话框/界面	
IDD_3DScanParas	3D 扫描重建设置参数对话框
IDD_DIFFERENTIALSETTINGSDLG	差动界面
IDD_DMDPATTERN	DMD、共聚焦、结构光界面
IDD_MAPVIEW	地图导航界面 MapViewDlg.h
IDD_MESSAGEDLG	显示剩余时间对话框
IDD_RESIZE	控制界面显示用 ResizeDlg.h
IDD_SCANCONTROL	地图、3D 的扫描界面
IDD_ShowCVDlg	图像均匀度调整界面
IDD_SI3DPANEL	差动视图 2，3D 图，调用 vtk SI3DPanel
IDD_SIIMAGEPANEL	差动视图 1，放置 SIImageWnd，结果图 SIImagePanel
IDD_SILINEPROFILEPANEL	差动视图 3，放置 HeatmapWnd，绘制测线 SILineProfilePanel
IDD_SIVIEWPANEL	差动视图 Table SIViewPanel
IDD_STAGE_DLG	控制轴界面
IDD_VIDEOCONFIG	相机控制界面

Icon
IDI_ICON_BOTTOM、……、IDI_ICON_Up

Menu	
IDR_MENU_HEATMAP	FitWindow、SaveFrame
IDR_MENU_SI	FitWindow、Start SI、Stop SI、SaveFrame、SaveProcessImg
IDR_MENU_SI_IMAGE	Defocus A 、Focus、Defocus B、FitWindow、SaveFrame 切换差动的三幅图
IDR_MENU1	FitWindow、SaveFrame
IDR_MENUConfocal	内容较多
IDR_ViedoViewMENU	FitWindow、SaveFrame、Alarm
IDR_ViewMutiLayer	FitWindow、SaveFrame、ReleaseAllImg（删除所有图）

6.2.2、输出函数

CConfocalUILib::GetDockPanel 获取控制面板	
DOCKPANEL_VIDEO DOCKPANEL_VIDEOEX	CVideoConfDlg::IDD IDD_VIDEOCONFIG , 控制摄像头
DOCKPANEL_SCANNERDLG	CScannerDlg::IDD IDD_SCANCONTROL , 扫描地图和 3D
DOCKPANEL_STAGECTL	CStageCtlDlg::IDD IDD_STAGE_DLG , 移动载物台
DOCKPANEL_DMDCTL	CDMDdnDlg::IDD, DMD IDD_DMDPATTERN , 用于控制 DMD
DOCKPANEL_MAPVIEWDLG	CMapViewDlg::IDD, 地图导航界面, 没 CResize IDD_MAPVIEW , 可以点选地图, 移动载物台
DOCKPANEL_DCAMCTL	空的
DOCKPANEL_DIFF	DifferentialSettingsDlg() IDD_DIFFERENTIALSETTINGSDLG , 差动
说明: 1、一般先创建 CResizeDlg::IDD, 再装载另一个窗口; 2、显示界面小于原本的对话框界面是, 有 CResize 会出现滚轮, 没 CResize 会盖住对话框超出部分!	

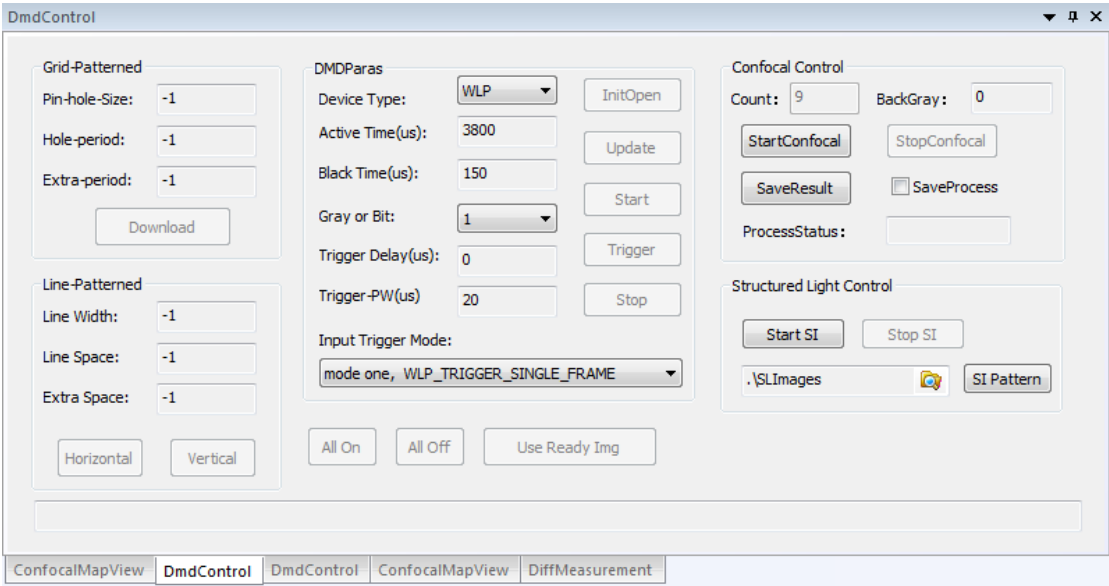
CConfocalUILib::GetViewPanel 获取视图	
VIEWPANEL_REALTIME VIEWPANEL_REALTIMEEX VideoView.h	"RealTime View", CVideoView() 实时扫描的图像
VIEWPANEL_CONFOCAL VIEWPANEL_CONFOCALEX MergeView.h	"Confocal View", CMergeView(); 共聚焦图像
VIEWPANEL_3D GLviewWnd.h	"3D View", CGLviewWnd() 包含 GIHeight.h 构建的 3D 图像。进行三维重建之后, 观察该图像时, 需用右键点击刷新
VIEWPANEL_COLOR ColorView.h	"Color View", CColorView() 预留
VIEWPANEL_ViewFocus WndBmp.h	"MutiFocuse View", CRenderWnd(); 全景的清晰图像。(部分位置仍不清晰, 主要是部分位置的光照条件不足等原因)
VIEWPANEL_3DRange ViewScan.h	"Multilayer View", ViewScan() Z 轴不同层次上的图像 通过 hFlushHeader, 刷新显示图像! m_ReBuildPro->ProcessImg(...,&hFlushHeader)
VIEWPANEL_SI	"SI View", SIView(); 仅是显示共聚焦图
VIEWPANEL_DIFF_MEASURE_VIEW	"DiffMeasure View", DiffMeasureView();

	在这个上面，会再叠加下面对话框 SIViewPanel 1、 SI3DPanel 2、 SIImagePanel（内放置 SIImageWnd） 3、 SILINEProfilePanel（内放置 HeatmapWnd）
说明：没有资源窗体，自己创建窗体！	

CConfocalUILib::GetMsgPanel 对话框	
MSG_TIME	CMsgViewBox() IDD_MESSAGEDLG ，显示计时结果
MSG_CALCV	CShowCVDlg() IDD_ShowCVDlg ，针对解决光照不均的问题

6.2.3、共焦（结构光）界面

先在停止相机的采集，然后再开始共聚焦或者 SL 构图！



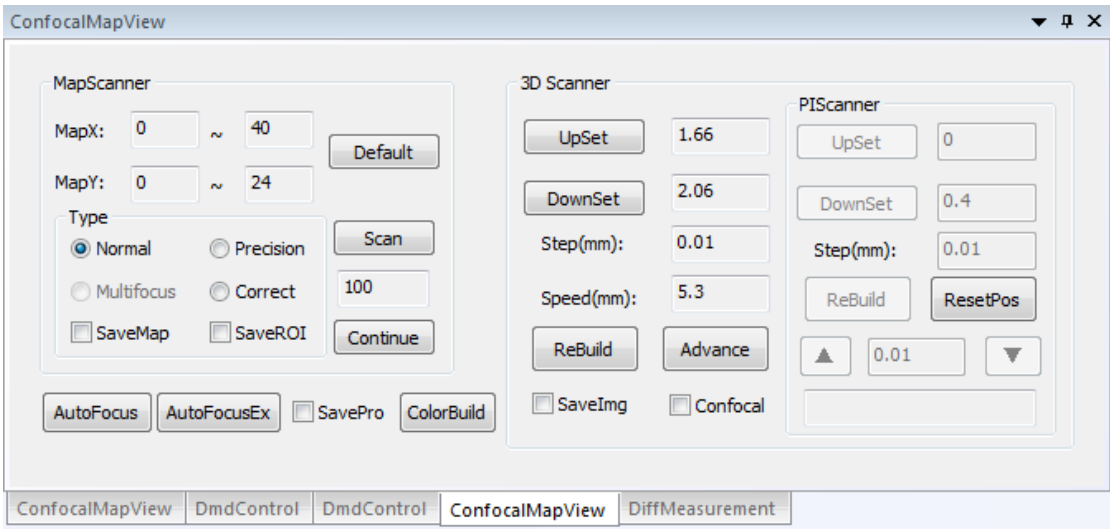
共聚焦设置界面（含 DMD）	
DMDdnDlg.h,	
	资源视图 IDD_DMDPATTERN
共聚焦启动	
CDMDdnDlg::OnBnClickedStartconbtn()	
1、 m_ConfocalCore->GetCurVideo()->SetTrigerMode(TRIGER_OUT);	
2、 this->Notify(this,NOTIFY_CONFOCAL_START,miPicCount,NULL,m_iBack...);	
CDMDdnDlg::OnBnClickedStopconbtn()	
1、 m_ConfocalCore->GetCurVideo()->SetTrigerMode(TRIGER_INTERAL);	
2、 this->Notify(this,NOTIFY_CONFOCAL_STOP);	
CDMDdnDlg::OnBnClickedSaveresbtn()	
1、 this->Notify(this,NOTIFY_CONFOCAL_SAVERESULT);	
共聚焦相关	
CDMDdnDlg::OnBnClickedSaveresbtn()	
1、 NOTIFY_CONFOCAL_SAVERESULT，通知线程库里面进行保存共聚焦结果	
CDMDdnDlg::OnBnClickedCheckprocess()	
1、 NOTIFY_CONFOCAL_SAVEPROCESS 通知线程库存储 Confocal 的过程图，	
会存在当前目录下的 ConfocalData 文件夹	
SI 结构光启动：	
CDMDdnDlg::OnBnClickedButtonStartsi()	
1、 m_ConfocalCore->GetCurVideo()->SetTrigerMode(TRIGER_OUT)	
2、 this->Notify(this,NOTIFY_SI_START,_si_frames, NULL, 0);	
CDMDdnDlg::OnBnClickedButtonStopsi()	

	1、 m_ConfocalCore->GetCurVideo()->SetTrigerMode(TRIGER_INTERAL); 2、 this->Notify(this, NOTIFY_SI_STOP); SI 配置流程: void CDMDdnDlg::OnBnClickedButtonSiPattern() 1、 打开路径下的 6 张图: horz_0、1、2.bmp; vert_0、1、2.bmp; 2、 下载这 6 张图, 再加 1 张黑图;
	消息响应 CDMDdnDlg::OnSubjectNotified(1、 Notify_WLPDMD_OPEN 接收 DMD 的消息 2、 Notify_WLPDMD_STOP 3、 Notify_WLPDMD_START 4、 NOTIFY_CONFOCAL_SAVEPROCESSEND 线程库通知存储 Confocal 的过程图结束 5、 NOTIFY_CONFOCAL_INPROCESS 线程库通知处理了一张图, 刷新显示图像进度 6、 NOTIFY_VIDEO_FORMAT_EXP (来自相机, 通知曝光时间、宽、高)

共聚焦视图界面	
	MergeView.h, 显示共聚焦图
	似乎是直接 void Renderer(HVideoHeader* pHeader,LPBYTE pBuffer);被调用

结构光视图	
	SIView.h
	好像就是共聚焦, 没什么特别的! 获取 HConfocalCore::RenderChain_SL 渲染链, 绑定本身, 用于显示 获取 PLUGIN_CONFOCALMODE 模块, 绑定消息, 似乎没用!
	通过右键菜单, 会发送下面的命令给 StructLightReconstruct.cpp this->Notify(this,NOTIFY_SI_START); this->Notify(this,NOTIFY_SI_STOP); this->Notify(this,NOTIFY_SI_SAVERESULT); this->Notify(this,NOTIFY_SI_SAVEPROCESS);

6.2.4、扫描 3D 界面

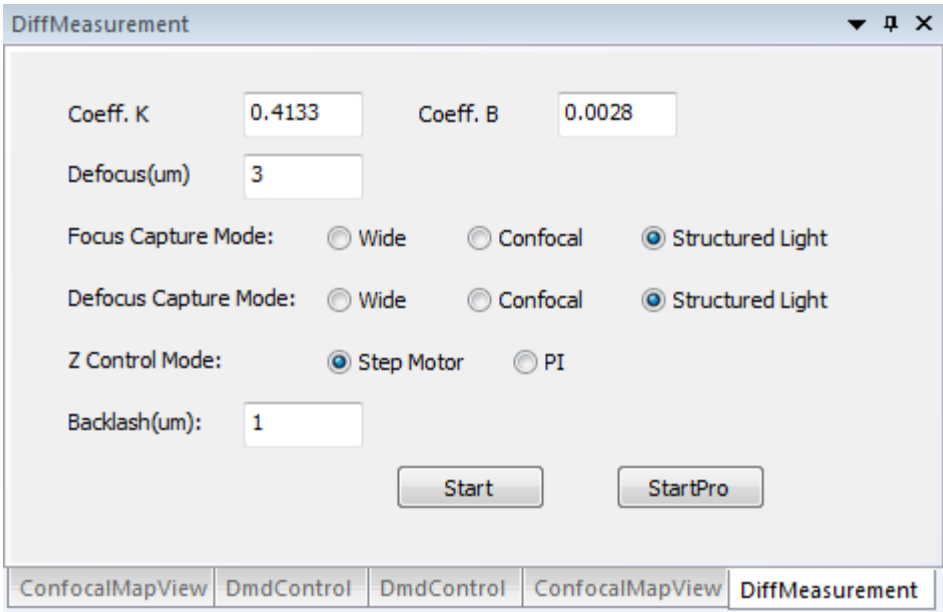


3D 扫描设置界面	
ScannerParas.h, 扫描 3D 的参数设置	
	资源视图 IDD_3DScanParas,
ScannerDlg.h, 包含地图扫描、两种 3D 扫描	
	资源视图 IDD_SCANCONTROL
1、	HConfocalPlug* tscan=m_ConfocalCore->GetPlugin(PLUGIN_3DScanner);获取线程;
2、	执行线程 tscan->Start(&bConfocal);

3D 扫描视图	
GLviewWnd.h (3D 视图)	
	CGLviewWnd::OnCreate()函数, 里面会初始化一些 OpenGL 的函数
	申请 CGIHeight m_GIHeight, 绘制 OpenGL 的函数
	CGLviewWnd::OnSubjectNotified;
	接收 PANEL_VIEW_3DCMOplete 这个消息
	调用 CGLviewWnd::OnRButtonUp
	载入图片, 并且显示 m_GIHeight.LoadFromBmp(L"../EpiResult//ResultHeight.bmp"); m_GIHeight.LoadGLTextures(L"../EpiResult//ResultPicture.bmp")
WndBmp.h (清晰图)	
	CRenderWnd::OnSubjectNotified
	接收 PANEL_VIEW_3DCMOplete 这个消息 HLoadBmp(&mLoadBuf,width,height,wbw,L"../EpiResult//ResultPicture.bmp");
ViewScan.h (不同层次的图)	
	接收 NOTIFY_3DSCAN_FLUSH 这个消息
	BOOL OnMouseWheel(...) 自动去 m_ReBuildPro->ProcessImg(&iCur,&iTotal,&hFlushHeader)读取刷新

		图片
--	--	----

6.2.5、差动界面



差动设置界面	
DifferentialSettingsDlg.h	
	使用资源文件：IDD_DIFFERENTIALSETTINGSDLG 构造函数里面，通过 m_ConfFile 读取所有的配置文件
DifferentialSettingsDlg::OnBnClickedButtonStart()	
	1、申请线程 HConfocalPlug* tscan = m_ConfocalCore->GetPlugin(PLUGIN_DIFFMEASURE); 2、保存参数 m_ConfFile.RecValue 3、设置线程参数 DiffMeasureParam dmp; tscan->Set(&dmp); 4、tscan->Start();启动线程

差动视图（差动三副图、高度剖面图、3D 视图） 七个头、源文件，四个资源视图	
DiffMeasureView.h, "DiffMeasure View"视图	
	申请（SIViewPanel.h）变量 SIViewPanel _panel; 资源视图 IDD_SIVIEWPANEL
	申请 SIImagePanel _img_panel; "Image",（即 SIImagePanel.h 加资源视图 IDD_SIIMAGEPANEL）
	申请 SIImageWnd _imgs_wnd SIImageWnd.h
	申请 SILineProfilePanel _line_panel; "Line Profile"（即 SILineProfilePanel.h 加资源视图 IDD_SILINEPROFILEPANEL）
	申请 HeatmapWnd _heatmap; HeatmapWnd.h; 含 “ChartClass\ChartCtrl.h”引用 "Winmm.lib"
	申请 SI3DPanel _3d_panel; "3D"（即 SI3DPanel.h 加资源视图

				IDD_SI3DPANEL)
消息传递				
				DiffMeasureView::OnSubjectNotified
				获取 NOTIFY_DIFF_MEASURE 消息
				从传参处复制三幅图到_imgs[...]里面 a、b、c test 处，判断是否从文件中去读取三幅图：HLoadBmp
				根据 b、c 去计算出三幅图到高度图_height_img 中 算法 1 数据类型，vector<double> _height_img 传递指针，double* h = _height_img.data()
				_panel.SetDiffData(), 把三幅图和高度图向下传递下去
				SIViewPanel::SetDiffData ()
				_img_panel.SetDiffData(w, h, b, a, c);
				SIIImagePanel::SetDiffData () 放置显示图像
				_imgs_wnd.SetDiffData(w, h, a, b, c);
				SIIImageWnd::SetDiffData(), 切换显示三幅图，可以绘制 ROI
				_line_panel.SetHeatmap(w, h, height_img)
				SILineProfilePanel::SetHeatmap()
				重新对高度图进行计算，得到新图 算法 2 _heatmap.Render(img.data(), _width, _height, 8);
				HeatmapWnd::Render(), 绘制曲线?
				重新采样，缩放图像，放大系数，计算得到 s_data _3d_panel.SetHeightData(sw, sh, s_data.data());
				SI3DPanel::SetHeightData(); Vtk 的显示

算法 1:

假设 P_1 代表焦后图， P_2 代表焦前图，针对每个像素

→ 如果 $P_1 + P_2$ 大于 18

■ K 大于 0 的话

◆ 像素值等于: $\frac{\frac{P_1 - P_2}{P_1 + P_2} - b}{k}$

■ 否则

◆ 像素值等于: $-\frac{P_1 - P_2}{P_1 + P_2} - b$

→ 否则，像素值等于 $(-0.5 - b) / k$

算法 2: 主要为了显示用！放大最高最低高度的比例！ P 为高度图

像素值等于: $p = \frac{P - P_{min}}{P_{max} - P_{min}} * 255$

6.3、 库程序 ConfocalScannerLib

区别相机的渲染对象:

- i. 滨松: m_VideoFormate.Vuser = 0;
- ii. 鑫图: m_VideoFormate.Vuser = 1;

6.3.1、 扫描 3D 线程

3D 扫描的过程

线程模块: **PLUGIN_3DScanner**

启动: C3DScanner::Start()

结束: 主动发送出: **PANEL_VIEW_3DCMOPLETE**

C3DScanner.h		
		C3DScanner::InitPlugin(m_AxisZ->GetSubject()->Attach(this) m_AxisPIZ 的移动, 直接延时了 200ms 后返回! m_ReBuildPro=m_ConfocalCore->GetCoreProcess(PROCESS_3DReBuild); 获取 3D 的算法库
		C3DScanner::Set(void* WParas) 保存传递进来的参数 BuildDirectory(strSavsPath);如果有需要保存图片, 则创建一个文件夹保存
		C3DScanner::OnSubjectNotified(...) 来自算法库 NOTIFY_RESULT_HEIGHT: 保存高度图 NOTIFY_RESULT_PIC: 保存结果图 NOTIFY_RESULT_3D: 发送 PANEL_VIEW_3DCMOPLETE 出去 来自 m_AxisZ NOTIFY_AXIS_MOVED 或者 NOTIFY_AXIS_POSITION: SetEvent(m_hMoveEvent);
		C3DScanner::Render(SetEvent(m_hCaptureEvent);
		C3DScanner::Start() 通过这个来启动任务 m_CaptureVideo.VnCount=m_ScanCount; 存入 3D 一共需要多少张图重构 一会通过 m_CaptureVideo 传递给算法库! 如果不是共聚焦, 绑定 GetRenderChain()->push_back(this); 启动线程 ScanProc: C3DScanner::ScanImage() 移动 m_AxisZ (等待移动结束), 抓拍图像 (有等待) 输出到 3D 库: m_ReBuildPro->GetVideoRender()->Renderer(...) 【m_CaptureVideo.VnCount 含有需要计算多少张图】 计数采集完图片, 退出线程 如果是共聚焦, 绑定 HConfocalCore::RenderChain_Confocal 与上面的区别是 m_AxisPIZ 没有反馈, 所以只有一个 sleep 等待时间!

6.3.2、扫描共聚焦线程

共聚焦扫描的过程

线程模块：

启动：发送 NOTIFY_CONFOCAL_START

结束：通过渲染链得到图片：RenderChain_Confocal

NOTIFY_CONFOCAL_STOP

NOTIFY_CONFOCAL_SAVERESULT

ConfocalMerge.h		
通过发送 NOTIFY_CONFOCAL_START 就可以启动共焦了！		
		CConfocalMerge::InitPlugin(
		启动 ImageProc m_RenderChain=m_ConfocalCore->GetRenderChain(HConfocalCore::RenderChain_Confocal); 设定渲染链
		启动 ImageProc 的线程 通过 m_hEvent 一直等待图像 等到了图像就调用 Img8_Max 累加图像 this->Notify(this,NOTIFY_CONFOCAL_INPROCESS,m_NowIndex); 刷新现在处理到第几张图片 处理完， m_RenderChain->Render(&tHeader,m_bmpResult);更新出去图片有保存，就保存一下图片
		CConfocalMerge::OnSubjectNotified(
		NOTIFY_CONFOCAL_START
		m_ConfocalCore->GetRenderChain()->push_back(this);【程序加了渲染链后，等到图片就会去设置 m_hEvent 事件】
		有传进来结构光要处理的图像数目、黑电平
		NOTIFY_CONFOCAL_STOP
		停止共焦
		NOTIFY_CONFOCAL_SAVERESULT
		创建文件夹， m_bSaveBmp=true; 存共聚焦结果图（在线程里面进行存储）
		NOTIFY_CONFOCAL_SAVEPROCESS
		bSaveConfocal=!bSaveConfocal; 存共聚焦过程图（在 Render 里面进行存储）

6.3.3、扫描结构光线程

SI 结构光扫描过程：

渲染链：RenderChain_SL

NOTIFY_SI_START

NOTIFY_SI_STOP

NOTIFY_SI_SAVERESULT

NOTIFY_SI_SAVEPROCESS

StructLightReconstruct.h		
通过发送 NOTIFY_SI_START 就可以启动任务了！		
		StructLightReconstruct::InitPlugin(
		启动 ImageProc
		m_RenderChain=m_ConfocalCore->GetRenderChain(HConfocalCore::RenderChain_Confocal); 设定渲染链
		启动 ImageProc 的线程
		1、通过 m_hEvent 一直等待图像，如果是 dark 图像，就删除，累计有效图像存放到 vector<vector<unsigned char>> _imgs;
		2、刷新现在处理到第几张图片
		this->Notify(this,NOTIFY_CONFOCAL_INPROCESS,m_NowIndex);
		3、合成图片 ReconstructImage(_imgs, m_bmpResult); 【3 或 6 张图】
		图像：horz_0、1、2.bmp; vert_0、1、2.bmp;
		a、b、c 代表三张图，A、B、C 代表三张图
		图像每个像素等于
		3 张图： $\frac{\sqrt{(2b-a-c)^2+(c-a)^2}}{2}$
		6 张图：两个上面的取平均；
		StructLightReconstruct::OnSubjectNotified(
		NOTIFY_SI_START
		m_ConfocalCore->GetRenderChain()->push_back(this);【程序加了渲染链后，等到图片就会去设置 m_hEvent 事件】
		有传进来结构光要处理的图像数目、黑电平
		NOTIFY_SI_STOP
		停止结构光图像
		NOTIFY_SI_SAVERESULT
		创建文件夹，m_bSaveBmp=true；存结果图（在线程里面进行存储）
		NOTIFY_SI_SAVEPROCESS
		bSaveConfocal=!bSaveConfocal；存共聚焦过程图（在 Render 里面进行存储）

6.3.4、扫描差动线程

操作	调用内容
Diff-DoubleCap	调用 DiffMeasurePro，双相机共焦 DiffMeasureProProc，串行处理线程 Cap0Proc、Cap1Proc、AllProc，并行处理线程，未调通 在“DiffMeasurePro::Start()”下的 bParallel 变量可控制并行与否 1、当前处于焦平面（焦平面没有相机采图）； 2、相机 1 采集非焦平面 1； 3、相机 2 采集非焦平面 2；
Diff-Test	调用 DiffMeasurePro， DiffMeasureProTestProc，测试线程 1、直接读取“DiffMeasureResult”文件夹下图做非焦面 2 图；
Diff-SingleCap	调用 DiffMeasure，做单相机配合 Z 轴的移动，达到差动目的 1、当前处于焦平面采图； 2、向上移动到非焦平面 1 采图；（移动量 defocus） 3、向下移动到非焦平面 2 采图；（移动量 defocus*2）
	算法部分，目前放置在了 UI 库中（DiffMeasureView::OnSubjectNotified），具体请见“差动界面”章节。

参数	说明
coeff_k	差动算法使用
coeff_b	差动算法使用
defocus	单相机差动时用，用于移动到非聚焦面
backlash	单相机差动时用，移动到非焦平面 2 时叠加背隙
focus_cap_mode	焦平面用的拍图模式，单相机差动时用
defocus_cap_mode	非焦平面用的拍图模式，双相机差动时调用
z_ctrl_mode	看使用的是 Motic 载物台或者是 PI 的载物台
bTest	用于控制 Diff-Test
iSleep	单相机差动时用，移动后延时采图
bFilter	增加了星星的滤波处理

手动启动共焦，然后再差动！

可选是 wide、confocal、SI

可选 Z 轴是 PTZ 还是载物台

差动扫描的过程

线程模块： **PLUGIN_DIFFMEASURE/ PLUGIN_DIFFMEASUREPRO**

启动：DiffMeasure/ DiffMeasurePro::Start(void* vParas)

结束：主动发送出： **NOTIFY_DIFF_MEASURE**

DiffMeasure.h 为示例说明	
	DiffMeasure::InitPlugin{

		m_AxisZ->GetSubject()->Attach(this) m_AxisPIZ 的移动，直接延时了 200ms 后返回
		DiffMeasure::Set(void* WParas)
		保存传递进来的参数 BuildDirectory(strSavsPath);如果有需要保存图片，则创建一个文件夹保存
		DiffMeasure::OnSubjectNotified(...)
		来自 m_AxisZ NOTIFY_AXIS_MOVED 或者 NOTIFY_AXIS_POSITION: SetEvent(m_hMoveEvent);
		DiffMeasure::Renderer{
		SetEvent(m_hCaptureEvent);
		DiffMeasure::Start{
		启动线程，最后调用 DiffMeasure::ScanImage() 1、根据 Z 轴类型，使用 m_AxisZ 或者 m_AxisPIZ; 2、根据 focus_cap_mode 取图类型，绑定 RenderChain_Normal、RenderChain_Confocal、RenderChain_SL; 3、采集聚焦图保存到_img_a 4、Z 向上移动离焦量 + 空回量；等待移动到位（事件 m_hMoveEvent） 5、Z 向下移动空回量；等待移动到位（事件 m_hMoveEvent） 6、根据 defocus_cap_mode 取图类型，绑定 RenderChain_Normal、RenderChain_Confocal、RenderChain_SL 7、延时（共聚焦延时 5s，其他延时 200ms） 8、采集聚焦图保存到_img_b【焦后图】 9、Z 向下移动 2 倍离焦量；等待移动到位（事件 m_hMoveEvent） 10、延时（共聚焦延时 5s，其他延时 200ms） 11、采集聚焦图保存到_img_c【焦前图】 12、保存图像，结束！ 13、Notify(this, NOTIFY_DIFF_MEASURE, 0, &dmr, 0, &dmp);参数里面含三张图！ 14、固定保存图片到 e 盘？！

6.4、主程序 DMDConfocal

CDMDConfocalApp::InitInstance(), 创建视图	
	系统自带的初始化, 创建多文档等...
	m_CofocalCoreLib.GetConfocalCore()->Init(), 初始化核心库【启动画面】
CMainFrame* pMainFrame = new CMainFrame;创建界面对话框	
	CMainFrame::CreateDockingWindows(), 创建界面
	pPanel->Create(.....)
int view_cnt = 10; 创建视图	
	GetNextDoc(), 使用 DMDConfocalDoc.h
	GetNextView(), 使用 DMDConfocalView.h
m_CofocalCoreLib.GetConfocalCore()->Init(¶ms), 初始化面板	

七、备注说明

- ➔ 结构光和共聚焦一样，通过发送消息给线程库，线程库绑定渲染链，然后就自动启动融合！融合结果通过渲染链发送出来！
- ➔ 3D 线程和差动一样，需要显性调用 **start** 启动！线程结束后的图像通过发送消息，发送出来！
 - 3D 线程可选 Z 轴类型、图像类型（普通、共焦）
 - 差动线程可选 Z 轴类型、图像类型（普通、共焦、结构光）
- ➔ 地图扫描如何保存、显示？
- ➔ 光照均匀度如何保存、应用？

总体修改	
	1、增加了 2 个 View（复用正常视图、共聚焦视图）； 2、增加了 1 个Dlg（复用控制相机界面）； 增加了 2 个任务（复用共焦任务，新建差动任务）
增加界面部分	
	1、在 DMDConfocal.cpp 中，增加：int view_cnt = 9； 2、在 MainFrm.cpp 中，增加：int PanelNum=7 3、在 ConfocalCoreModule.cpp，去增加渲染链与相机 a) 渲染链：m_RenderChainEx、m_ConfocalChainEx b) 渲染链枚举：RenderChain_NormalEx、RenderChain_ConfocalEx c) 相机：m_VideoManageEx、m_VideoDeviceEx、 d) 相机接口：GetVideoManagerEx()、GetCurVideoEx() 4、在 ConfocalCoreModule.cpp，去增加新建界面与任务的初始化 a) ConfocalCoreModule::Init(中，去增加查找相机、绑定渲染链与相机 b) GetViewPanel(VIEWPANEL_REALTIMEEX)->InitPanel(this,&bValue); c) GetDockablePanel(DOCKPANEL_VIDEOEX)->InitPanel(this, &bValue); d) GetViewPanel(VIEWPANEL_CONFOCALEX)->InitPanel(this, &bValue); e) GetPlugin(PLUGIN_CONFOCALMODEEX)->InitPlugin(this,&bValue); f) GetPlugin(PLUGIN_DIFFMEASUREPRO)->InitPlugin(this);
复用视图界面 VideoView.h	
	1、增加枚举 VIEWPANEL_REALTIMEEX 2、没有发出去消息； 3、增加绑定响应消息 a) CStageCtlDlg::InitPanel（）绑定 NOTIFY_MAPROISIZE_FLUSH b) CShowCVDlg::InitPanel（）绑定 NOTIFY_CVCaI_FLUSH 4、增加绑定渲染链 a) 根据额外的标志 Ex ： RenderChain_NormalEx
复用共焦视图 MergeView.h	
	1、增加枚举 VIEWPANEL_CONFOCALEX 2、响应消息，空； 3、发送消息，都是给任务 ConfocalMerge.cpp（共焦任务）响应； a) NOTIFY_CONFOCAL_START

	b) NOTIFY_CONFOCAL_STOP c) NOTIFY_CONFOCAL_SAVERESULT d) NOTIFY_CONFOCAL_SAVEPROCESS 4、增加发送消息绑定：CMergeView::InitPanel(, PLUGIN_CONFOCALMODEEX 5、增加绑定渲染链：CMergeView::InitPanel(a) 根据额外的标志 Ex : RenderChain_ConfocalEx
复用控制相机界面 VideoConfDlg.h	
	1、增加枚举 DOCKPANEL_VIDEOEX 2、CConfocalUILib::GetDockPanel()中，去修改文件夹名称：VideoControl Ex; 3、响应的消息是相机设备发送过来的; 4、响应消息绑定是在：CVideoConfDlg::InitPanel 里面！ 5、发送消息，NOTIFY_VIDEO_FORMAT_EXP，更改曝光时间、宽和高都发送 6、发送消息的绑定，在 CDMDdnDlg::InitPanel(里面绑定，不需增加
复用共焦任务 ConfocalMerge.h	
	1、增加枚举 PLUGIN_CONFOCALMODEEX 2、绑定渲染链：CConfocalMerge::OnSubjectNotified(里面; 3、设定渲染链：CConfocalMerge::InitPlugin(, m_RenderChain=……; 4、响应消息 a) NOTIFY_CONFOCAL_START; (会去绑定渲染链!) b) NOTIFY_CONFOCAL_STOP c) NOTIFY_CONFOCAL_SAVERESULT 保存路径有替换，CConfocalMerge::Save_Result() d) NOTIFY_CONFOCAL_SAVEPROCESS 保存路径有替换，CConfocalMerge::Renderer(5、响应消息的绑定：CMergeView::InitPanel、CDMDdnDlg::InitPanel(6、发送消息 a) NOTIFY_CONFOCAL_INPROCESS, NOTIFY_CONFOCAL_SAVEPROCESSEND, 影响 DMDdnDlg，用来修改 UI 7、发送消息的绑定，CDMDdnDlg::InitPanel(, 不需增加
新建任务 DiffMeasurePro	
	1、增加枚举 PLUGIN_DIFFMEASUREPRO; 2、接收消息：空 3、发送消息：NOTIFY_DIFF_MEASURE; 4、发送消息绑定：DiffMeasureView::InitPanel(; 5、可接收普通图像、共焦图像进行差动 a) 普通图像：.Vuser = 0 或者 1 分别代表滨松和鑫图; b) 共焦图像：.Vuser = 0 或者 1 分别代表滨松和鑫图的共焦;